

SOFTWARE REQUIREMENTS SPECIFICATION

IEEE 830 / ISO-IEC-IEEE 29148 Compliant Document

Prediction of Protein Secondary Structure

A Machine Learning and Deep Learning Approach

Dataset: "Protein Secondary Structure" (Kaggle,alfrandom)

Document Version	1.0
Document Status	Final
Classification	Academic / Research Project
Domain	Bioinformatics — Computational Biology
Prepared By	Tayyaba Sadaqat
Date of Issue	24 August 2026

Table of Contents

- Revision History 4**
- 1 Introduction 4**
 - 1.1 Purpose 4
 - 1.2 Document Conventions 4
 - 1.3 Intended Audience and Reading Suggestions 4
 - 1.4 Project Scope 5
 - 1.5 Definitions, Acronyms, and Abbreviations 5
 - 1.6 References 6
- 2 Overall Description 6**
 - 2.1 Product Perspective 6
 - 2.2 Product Functions 6
 - 2.3 User Classes and Characteristics 7
 - 2.4 Operating Environment 7
 - 2.5 Design and Implementation Constraints 7
 - 2.6 Assumptions and Dependencies 7
- 3 Dataset Description 8**
 - 3.1 Data Source 8
 - 3.2 Data Schema 8
 - 3.3 Data Preprocessing Requirements 8
 - 3.4 Train / Validation / Test Partitioning 9
- 4 System Features (Functional Requirements) 9**
 - 4.1 Data Ingestion and Preprocessing 9
 - 4.2 Feature Engineering 9
 - 4.3 Machine Learning Modeling Module 10
 - 4.4 Deep Learning Modeling Module 10
 - 4.5 Model Evaluation and Comparison Module 10
 - 4.6 Visualization and Reporting Module 10
 - 4.7 Model Persistence Module 11
- 5 External Interface Requirements 11**
 - 5.1 User Interfaces 11
 - 5.2 Hardware Interfaces 11
 - 5.3 Software Interfaces 11
 - 5.4 Communication Interfaces 12
- 6 Non-Functional Requirements 12**
 - 6.1 Performance Requirements 12
 - 6.2 Reliability and Robustness 12

6.3 Usability and Maintainability.....	12
6.4 Portability	12
6.5 Security and Data Integrity.....	13
7 System Architecture Overview	13
8 Other Requirements.....	14
8.1 Legal and Licensing	14
8.2 Ethical Considerations	14
Appendix A — Glossary of Model Terms.....	14
Appendix B — Requirements Traceability Matrix	15

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional, non-functional, data, and interface requirements for a project whose sole objective is to design, implement, train, and evaluate Machine Learning (ML) and Deep Learning (DL) models that predict the secondary structure of proteins from their primary amino acid sequence. The document is intended to serve as the single authoritative reference for the scope, behavior, and quality attributes of the system throughout its design, development, testing, and evaluation phases.

The document follows the structure and conventions recommended by the IEEE 830-1998 and ISO/IEC/IEEE 29148 standards for software requirements specifications, adapted to the context of a data-science and modeling project rather than a traditional transactional application.

1.2 Document Conventions

Requirement statements use the keyword “shall” to denote a mandatory requirement, “should” to denote a recommended requirement, and “may” to denote an optional requirement. Each functional requirement is uniquely identified using the prefix FR-xx, and each non-functional requirement uses the prefix NFR-xx, enabling full traceability. Section headings are numbered hierarchically; tables and requirement identifiers are referenced consistently across the document.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following audiences, each of whom may focus on a specific subset of the material:

- Project developers and data scientists — Sections 3, 4, 5, and 7 for implementation guidance.
- Course instructors / project evaluators — Sections 1, 2, and 6 for scope, objectives, and quality criteria.
- Quality reviewers / testers — Section 6 and Appendix C for verification criteria and traceability.
- Future maintainers — The full document, with particular attention to Sections 3 and 7 (data and architecture).

1.4 Project Scope

The system shall be strictly limited to the application of Machine Learning and Deep Learning techniques to a single, well-defined dataset: “Protein Secondary Structure” dataset published on Kaggle by the contributor “alfrandom.” The project explicitly excludes any other Kaggle datasets, Kaggle competition submission or leaderboard participation, external protein databases, and any functionality unrelated to the modeling pipeline described herein (e.g., web scraping, general-purpose bioinformatics tooling, or unrelated Kaggle platform features).

In scope:

- Ingestion and preprocessing of the specified dataset (training file and CB513-derived independent test file).
- Encoding of amino-acid sequences into numerical/categorical representations suitable for ML and DL algorithms.
- Development of classical Machine Learning models (e.g., Logistic Regression, Support Vector Machine, Random Forest, Gradient Boosting, k-Nearest Neighbors) for secondary-structure classification.
- Development of Deep Learning models (e.g., 1D-CNN, LSTM/Bi-LSTM, GRU, and Transformer-encoder architectures) for the same task.
- Quantitative evaluation, comparison, and visualization of model performance using standard classification metrics.
- Persistence of trained models and generation of a reproducible experimental report.

Out of scope:

- Any dataset other than the one explicitly named above.
- Kaggle competition entry, notebook publishing, leaderboard scoring, or community features.
- Three-dimensional tertiary structure prediction, molecular docking, or protein folding simulation.
- Production deployment, public API hosting, or integration with external laboratory systems.

1.5 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
ML	Machine Learning — algorithms that learn statistical patterns from tabular/engineered features
DL	Deep Learning — neural-network-based algorithms that learn hierarchical representations directly from sequence data
Q3	Three-state secondary structure classification scheme: Helix (H), Strand/Sheet (E), Coil (C)
Q8	Eight-state secondary structure classification scheme (DSSP): H, G, I, E, B, T, S, C
SST3 / SST8	Dataset columns holding the ground-truth 3-state and 8-state secondary structure labels
FASTA / seq	The linear, one-letter-coded amino acid sequence of a protein chain
PSSM	Position-Specific Scoring Matrix — an optional evolutionary feature representation
CB513	A widely used, non-redundant benchmark test set for secondary structure prediction
CNN	Convolutional Neural Network
LSTM / Bi-LSTM	(Bidirectional) Long Short-Term Memory recurrent neural network

Term	Definition
One-Hot Encoding	A binary vector representation in which each amino acid is mapped to a unique unit vector
Q3/Q8 Accuracy	Per-residue classification accuracy under the 3-state or 8-state scheme
EDA	Exploratory Data Analysis

1.6 References

- Kaggle Dataset: “Protein Secondary Structure,” contributor alfrandom — <https://www.kaggle.com/datasets/alfrandom/protein-secondary-structure>
- Cuff, J.A. & Barton, G.J. (1999). “Evaluation and improvement of multiple sequence methods for protein secondary structure prediction” (origin of the CB513 benchmark).
- Zhou, J. & Troyanskaya, O. (2014). “Deep Supervised and Convolutional Generative Stochastic Network for Protein Secondary Structure Prediction,” ICML.
- Kabsch, W. & Sander, C. (1983). “Dictionary of protein secondary structure: pattern recognition of hydrogen-bonded and geometrical features” (DSSP method, source of SST8 labels).
- IEEE Std 830-1998, “IEEE Recommended Practice for Software Requirements Specifications.”
- ISO/IEC/IEEE 29148:2018, “Systems and software engineering — Life cycle processes — Requirements engineering.”

2 Overall Description

2.1 Product Perspective

The product is a self-contained, offline experimentation pipeline rather than a networked or client-server application. It consumes a static, locally stored CSV dataset and produces trained model artifacts, evaluation reports, and visualizations. It has no dependency on any other Anthropic, Kaggle, or third-party production system; the only external artifact it depends on is the dataset file(s) downloaded once from Kaggle.

2.2 Product Functions

At a high level, the system shall provide the following capability groups, each elaborated as a formal feature in Section 4:

- Automated loading, validation, and cleaning of the raw dataset.
- Sequence-to-feature transformation (encoding, windowing, padding).
- Training of multiple ML classifiers on engineered tabular features.
- Training of multiple DL architectures directly on encoded sequences.
- Unified evaluation using per-residue and per-protein metrics.
- Comparative visualization and export of a results report.

2.3 User Classes and Characteristics

User Class	Characteristics	Primary Interaction
Model Developer / Student	Proficient in Python, ML/DL libraries; primary implementer of the pipeline	Writes and runs notebooks/scripts; tunes hyperparameters
Evaluator / Instructor	Domain-aware reviewer assessing methodology and results, not necessarily writing code	Reviews generated reports, metrics tables, and visualizations
Future Maintainer	Developer extending the project later (e.g., adding a new architecture)	Reads code, SRS, and architecture documentation before modification

2.4 Operating Environment

- Operating System: Windows 10/11, Linux (Ubuntu 20.04+), or macOS 12+.
- Language/Runtime: Python 3.9 or later.
- Core Libraries: NumPy, Pandas, Scikit-learn, TensorFlow/Keras or PyTorch, Matplotlib/Seaborn.
- Development Environment: Jupyter Notebook, JupyterLab, or an equivalent IDE (VS Code, PyCharm).
- Hardware: CPU execution is sufficient for the ML models; a CUDA-capable GPU (≥ 4 GB VRAM) is recommended, though not mandatory, for efficient DL training.

2.5 Design and Implementation Constraints

- The project shall use only the specified Kaggle dataset; no supplementary sequence, structural, or evolutionary databases (e.g., UniRef, PDB, BLAST profiles) shall be introduced unless derivable from the given files.
- The project shall not include any Kaggle-competition-specific mechanism (submission files formatted for leaderboard scoring, kernel publishing, etc.).
- All experiments shall be reproducible via fixed random seeds and a documented environment (requirements.txt / environment.yml).
- Class imbalance inherent to secondary-structure labels (coil residues dominate) shall be explicitly addressed in the modeling strategy.

2.6 Assumptions and Dependencies

- The dataset file(s) are assumed to be downloaded manually by the user from Kaggle and placed in a local data/ directory prior to execution.
- The dataset schema (columns pdb_id, chain_code, seq, sst8, sst3, len, has_nonstd_aa) is assumed stable and consistent with the version described in Section 3.
- Sufficient local compute resources (RAM ≥ 8 GB) are assumed to be available for DL model training on the full dataset.

3 Dataset Description

3.1 Data Source

The system shall exclusively use the “Protein Secondary Structure” dataset (Kaggle, contributor alfrandom), an update and re-curation of the classical CullPDB / CB513 secondary-structure benchmark. The dataset is derived from the RCSB Protein Data Bank (PDB) and processed with the PISCES culling server and the DSSP algorithm to assign per-residue structural labels. It is distributed as CSV files containing a large, non-redundant training set together with an independent CB513-derived test set traditionally used for benchmarking.

3.2 Data Schema

The system shall treat each row of the dataset as one protein chain, with the following fields:

Field	Type	Description
pdb_id	String	Unique four-character Protein Data Bank identifier of the source structure
chain_code	String	Chain identifier within the PDB entry (e.g., A, B)
seq	String	Primary amino-acid sequence of the chain, one-letter code
sst8	String	Ground-truth secondary structure label per residue, 8-state DSSP scheme
sst3	String	Ground-truth secondary structure label per residue, reduced 3-state scheme (H / E / C)
len	Integer	Sequence length (number of residues)
has_nonstd_aa	Boolean	Flag indicating presence of non-standard amino-acid codes (e.g., X, B, Z, U)

3.3 Data Preprocessing Requirements

The system shall perform the following preprocessing operations prior to model training, each traceable to a functional requirement in Section 4:

1. Remove or flag records with `has_nonstd_aa = True` where non-standard residues cannot be meaningfully encoded, or map them to a designated “unknown” token.
2. Validate that `seq`, `sst8`, and `sst3` have matching lengths for every record; discard or repair malformed rows.
3. Optionally collapse the 8-state label set to the 3-state scheme (or use the provided `sst3` column directly) to support both Q3 and Q8 experimentation tracks.
4. Encode amino-acid residues using one-hot or learned embedding representations for DL models, and using engineered window/statistical features for ML models.
5. Apply padding/truncation to a configurable maximum sequence length for batch-based DL training, with an accompanying attention/validity mask.

3.4 Train / Validation / Test Partitioning

The system shall reserve the CB513-derived subset (or an equivalent held-out partition explicitly identified in the dataset) exclusively for final, independent testing. From the remaining training data, the system shall

further allocate a stratified validation split (recommended 85% train / 15% validation) for hyperparameter tuning and early stopping, ensuring that no protein chain appears in more than one partition.

4 System Features (Functional Requirements)

Each feature below is stated as a discrete capability of the system, accompanied by its constituent functional requirements and an assigned priority (High / Medium / Low) reflecting its necessity to satisfy the project scope defined in Section 1.4.

4.1 Data Ingestion and Preprocessing

FR-01 Dataset Loading *Priority: High*

The system shall load the training and CB513 test CSV files from a configurable local path into memory-efficient data structures (Pandas DataFrames), reporting row counts, column types, and basic integrity checks upon load.

FR-02 Data Cleaning *Priority: High*

The system shall detect and handle missing values, mismatched sequence/label lengths, and non-standard amino-acid codes according to a documented, configurable policy (drop, flag, or map to 'unknown').

FR-03 Exploratory Data Analysis *Priority: Medium*

The system shall generate descriptive statistics and visualizations of sequence-length distribution, amino-acid frequency, and secondary-structure class balance (Q3 and Q8) prior to modeling.

4.2 Feature Engineering

FR-04 Sequence Encoding for ML *Priority: High*

The system shall transform each residue (with a configurable sliding window of neighboring residues) into a fixed-length numerical feature vector suitable for classical ML classifiers, including one-hot residue identity and local compositional statistics.

FR-05 Sequence Encoding for DL *Priority: High*

The system shall transform each full sequence into a padded, integer-tokenized or one-hot tensor representation, together with a validity mask, suitable for direct input to neural-network architectures.

4.3 Machine Learning Modeling Module

FR-06 Classical ML Model Training *Priority: High*

The system shall train and expose configuration for at least four classical ML classifiers — e.g., Logistic Regression, Support Vector Machine, Random Forest, and Gradient Boosting (e.g., XGBoost/LightGBM) — on the engineered residue-window features.

FR-07 Hyperparameter Tuning (ML) *Priority: Medium*

The system shall support systematic hyperparameter search (grid search, random search, or equivalent) with cross-validation for each ML model, recording the best configuration and validation score.

4.4 Deep Learning Modeling Module

FR-08 Baseline DL Models *Priority: High*

The system shall implement and train at least three DL architectures suited to sequence labeling: a 1D Convolutional Neural Network, a Bidirectional LSTM/GRU recurrent network, and a Transformer-encoder-based model.

FR-09 Training Control *Priority: High*

The system shall support configurable training parameters (batch size, learning rate, optimizer, number of epochs) and shall implement early stopping and checkpointing based on validation loss/accuracy.

FR-10 Regularization *Priority: Medium*

The system shall provide dropout, L2 weight regularization, and class-weighting mechanisms to mitigate overfitting and label imbalance during DL training.

4.5 Model Evaluation and Comparison Module

FR-11 Metric Computation *Priority: High*

The system shall compute per-residue accuracy (Q3 and Q8), macro/weighted precision, recall, and F1-score, and a confusion matrix for every trained model on the held-out CB513 test set.

FR-12 Cross-Model Comparison *Priority: High*

The system shall aggregate all model results into a unified comparison table, ranking ML and DL models on the same evaluation metrics under identical test conditions.

4.6 Visualization and Reporting Module

FR-13 Result Visualization *Priority: Medium*

The system shall generate accuracy/loss training curves, confusion-matrix heatmaps, and per-class performance bar charts for each trained model.

FR-14 Report Export *Priority: Medium*

The system shall export a consolidated results report (tables and figures) in a reproducible, shareable format (e.g., notebook, PDF, or Markdown) summarizing methodology, configuration, and findings.

4.7 Model Persistence Module

FR-15 Model Saving and Loading *Priority: Low*

The system shall serialize trained ML models (e.g., via joblib/pickle) and DL models (e.g., via native framework checkpoint formats) to disk, and shall support reloading a saved model for inference without retraining.

5 External Interface Requirements

5.1 User Interfaces

The primary interface shall be a set of documented Jupyter/Python notebooks or command-line scripts exposing clearly named configuration parameters (dataset path, model choice, hyperparameters). Console/log output shall report progress, elapsed time, and key metrics at each pipeline stage. No graphical desktop or web application is required by the scope of this project.

5.2 Hardware Interfaces

The system shall interface with local host hardware only: CPU, system RAM, local disk storage, and — optionally, when available — a CUDA-capable GPU accessed through the deep learning framework's standard device APIs. No specialized or laboratory hardware interfaces are required.

5.3 Software Interfaces

Software Component	Purpose	Interface Type
Pandas / NumPy	Data loading, cleaning, numerical feature computation	Python API (in-process library)
Scikit-learn	Classical ML model training, cross-validation, metrics	Python API (in-process library)
TensorFlow/Keras or PyTorch	Deep learning model definition and training	Python API (in-process library, optional GPU backend)
Matplotlib / Seaborn	Chart and figure generation for the report	Python API (in-process library)

Software Component	Purpose	Interface Type
Local File System	Storage of dataset CSVs, saved models, and generated reports	File I/O

5.4 Communication Interfaces

The system shall not require any network communication for its core functionality once the dataset has been downloaded locally. No client-server, REST, or messaging interfaces are within scope.

6 Non-Functional Requirements

6.1 Performance Requirements

ID	Requirement
NFR-01	The full preprocessing pipeline shall process the entire training dataset in under 5 minutes on a standard quad-core CPU with 8 GB RAM.
NFR-02	Each classical ML model shall complete training and hyperparameter search within 30 minutes on the reference hardware described in Section 2.4.
NFR-03	Each DL model shall complete a full training run (subject to early stopping) within 3 hours on the reference hardware, or under 30 minutes when a compatible GPU is available.
NFR-04	Inference latency for a single protein sequence (≤ 700 residues) shall not exceed 2 seconds for any trained model.

6.2 Reliability and Robustness

- NFR-05 — The system shall handle malformed or unexpected rows in the dataset gracefully, logging a warning rather than terminating the pipeline.
- NFR-06 — Training procedures shall be deterministic given a fixed random seed, to within the inherent non-determinism of GPU floating-point operations.

6.3 Usability and Maintainability

- NFR-07 — All modules (preprocessing, ML, DL, evaluation, reporting) shall be organized as clearly separated, independently runnable components with documented inputs and outputs.
- NFR-08 — Code shall include inline documentation and a top-level README describing setup, dataset placement, and execution order.
- NFR-09 — Configuration values (paths, hyperparameters, model choice) shall be centralized rather than hard-coded across multiple files.

6.4 Portability

- NFR-10 — The system shall run unmodified on Windows, Linux, and macOS platforms supporting Python 3.9+.

- NFR-11 — The system shall function correctly with or without an available GPU, automatically falling back to CPU execution.

6.5 Security and Data Integrity

- NFR-12 — The system shall not transmit dataset content, model weights, or results to any external service.
- NFR-13 — The system shall verify (e.g., via row/column count checks) that a loaded dataset matches the expected schema before proceeding, to prevent silent corruption of results.

7 System Architecture Overview

The system shall be organized as a linear, modular pipeline in which each stage consumes the output of the previous stage and produces a well-defined artifact. The conceptual flow is summarized below.

Stage	Input	Output
1. Data Ingestion	Raw Kaggle CSV files (train, CB513 test)	Validated in-memory DataFrames
2. Preprocessing & Cleaning	Validated DataFrames	Cleaned sequences and aligned labels
3. Feature Engineering	Cleaned sequences	ML feature matrix + DL tensor representations
4. ML Model Training	ML feature matrix	Trained classical models + validation scores
5. DL Model Training	DL tensors	Trained neural network checkpoints
6. Evaluation	Trained models + CB513 test set	Metrics tables and confusion matrices
7. Reporting & Persistence	Metrics, figures, model artifacts	Consolidated report + saved models

This layered structure ensures that the Machine Learning and Deep Learning tracks share a common, validated data foundation (Stages 1–2) while diverging only where algorithmically necessary (Stages 3–5), and reconverge for a fair, unified comparison (Stage 6–7), directly satisfying the project's exclusive ML/DL scope stated in Section 1.4.

8 Other Requirements

8.1 Legal and Licensing

The dataset shall be used strictly in accordance with the license terms published on its Kaggle dataset page. Any results, figures, or derived artifacts produced by the system shall attribute the original dataset source (Kaggle contributor alfrandom, derived from RCSB PDB data).

8.2 Ethical Considerations

As the project involves only publicly available, de-identified structural biology data with no human-subject or personally identifiable information, no additional ethical review is required. Findings shall be reported honestly, including model limitations and failure cases, to avoid overstating predictive reliability for any downstream biological interpretation.

Appendix A — Glossary of Model Terms

Term	Definition
Secondary Structure	Local, regular conformation of a protein backbone (helix, strand, coil) stabilized by hydrogen bonds
Residue	A single amino-acid unit within a protein sequence
Overfitting	A model that fits training data well but generalizes poorly to unseen sequences
Early Stopping	Halting training once validation performance stops improving, to prevent overfitting
Confusion Matrix	A tabular summary of predicted vs. actual class labels used to assess classifier performance
Macro F1-score	The unweighted average of per-class F1-scores, sensitive to performance on minority classes

Appendix B — Requirements Traceability Matrix

Req. ID	Description	Source Section	Verification Method
FR-01	Dataset Loading	4.1	Unit test on file load + schema check
FR-02	Data Cleaning	4.1	Test on injected malformed records
FR-04	ML Feature Encoding	4.2	Manual inspection of feature matrix shape/values
FR-05	DL Sequence Encoding	4.2	Manual inspection of tensor shape/padding mask
FR-06	Classical ML Training	4.3	Validation-set score review

Req. ID	Description	Source Section	Verification Method
FR-08	Baseline DL Models	4.4	Training-curve review + validation score
FR-11	Metric Computation	4.5	Cross-check metrics against sklearn reference implementation
FR-12	Cross-Model Comparison	4.5	Review of consolidated comparison table
FR-14	Report Export	4.6	Manual review of exported report artifact
NFR-01–04	Performance Targets	6.1	Timed execution on reference hardware

End of Document